# FAR ADDRESSES & NEAR ADDRESSES

- complete specification => FAR ADDRESS
- near address is an incomplete address
- if you want to go to a diff. code segment you have to specify „FAR", otherwise it's near as implicit.

- complete address specification:
  $s_3 s_2 \wedge s_1 s_0$ ! offset_specification, $s_i$ - constant.
- pointer variable must be on at least 6 bytes (or 8)
  (4 bytes offset + 2 bytes base)
- 2) segment_register : offset_specification
  CS, DS, SS, ES, FS, GS
  code seg, data seg, stack,

Little endian applies only to memory.

12 34 56 78 h => 78h 56h 34h 12h

1 byte => NONSense lil' endian

at least 2 bytes => yes ✓

word, dword, qword

The most useful bytes are the ones from the end (since they argued that most of the L. higher ones are 0). [the numbers are small]

instr_name [dest], [source] ← optionality, also explicit

i) immediate mode (ex: mov eax, 17)

ii) register mode

iii) memory addressing mode

↑ possibilities for the operands.

Segment part: Operating Sys.

Offset part:

- For iii) it's offset must obey this specification

based addressing (register) | indexed addr. (register) | direct addressing

offset_spec = [base] + [index * scale] + [constant]

indirect addressing

- As a base, you can use any one of the general 8 registers
- As index 8 registers w/o ESP

- scale with 1, 2, 4, 8

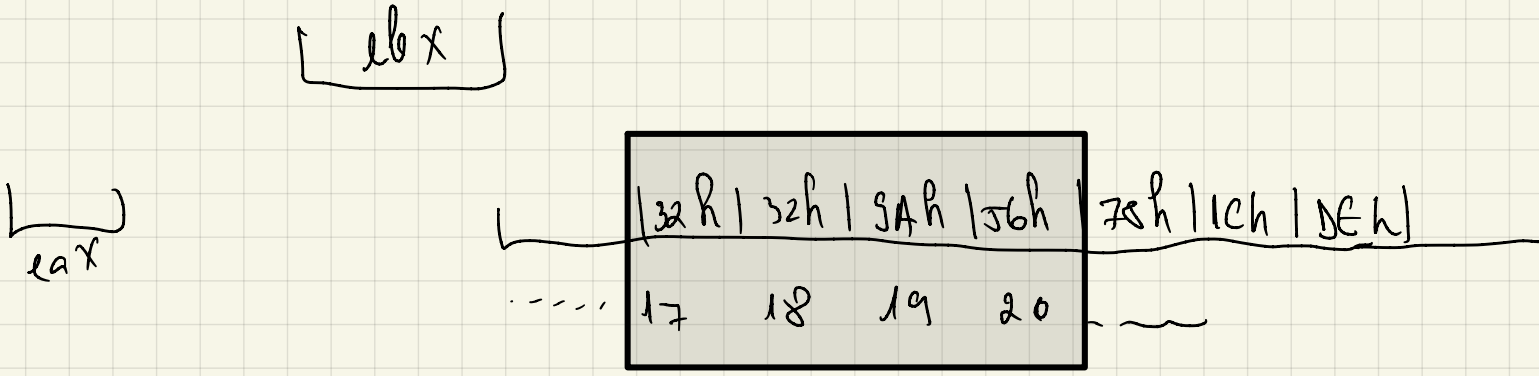
A REGISTER IS NOT A MEMORY EL.

mov ebx, 17

mov eax, ebx ; eax = 17

mov eax, [ebx] ; EAX = 3232 9A56 h

[ ] operator
* dereferencing. } get the value




mov eax, ebx+2 - syntax error

mov ecx, [ebx+2] ; from 19. = 1C78 569A h

mov eax, [ebx + 2*esp - 7] ; syntax error.

mov eax, [ebx + esp] ; works. (bc. no scale)

Base here

mov ebx, 17

mov eax, [ebx*2 - 7] ✓

mov eax, [ebx*3 - 7] ✓

$$[ebx + ebx*2 - 7]$$

mov eax, [ebx - edx] ✗ syntax error

mov [ebx + 2 - 7], eax

↑ overwrites first 4 bytes from that position

In any prog. lang., any declared variable will have a fixed address. (the offset, the segment you don't know as only). It's offset is always a constant determinable at assembly / compile time.

.exe contains [code segment]
[data segment]

⇒ mov eax, [a] ⇐ constant address.

Pointer arithmetic (only inside the memory bounds)

- we can't add to pointers!
- we can't multiply / divide pointers

- substraction (from 2 addr.) gives the diff bytes offset.
- we can ADD / SUBSTR. A CONSTANT ✓

$$a[7] = *(a+7) = *(7+a) = 7[a]$$

$i = i+1$

address of i (left hand side is always a pointer)

LHS RHS

L-value R-value

symbol := expression

identifier := expression

address_computation - expression := expression

mov [ebx + edx*4 - 7], eax

LHS, not a symbol

Basic elements of AST